

Single Zone IIoT Identity Management Framework

Implementation and evaluation report

Muhammad Hassaan Siddiqui | Muhammad Huzaifa | Irbaz

30 September 2026 | One zone, one fog, five simulated devices

Summary

We implemented a single-zone device identity lifecycle in Python. A fog node accepts devices only after a PSK check and an ECC proof of key ownership. While registration is open, a fog-signed temporary token grants limited access. Once a batch is finalized, devices use an epoch-bound Merkle proof and a fresh signature for resource requests. Revocation and resource rules can still deny a device whose old proof is mathematically valid.

The complete localhost demonstration registered five devices over a verified TLS connection, allowed a sensor to write readings with a temporary token, and allowed a motor controller to stop the production line only after permanent registration. Security demonstrations rejected replay, changed claims, stolen tokens, expiry, and revoked identities. A five-size benchmark produced mean measurements and graphs.

System architecture



Figure 1 One fog mediates registration, tokens, proofs, and authorization

Component	Responsibility
Devices	Generate DID and P-256 key; prove PSK and sign fresh requests.
Fog	Keep trusted roles and PSKs; sign tokens; build roots; check access.
Local registry	Retain epoch roots, proof packages, and revocation lists.

Technologies and primitives

Python standard-library ssl, hashlib and hmac provide TLS transport, hashing and PSK challenge response. The cryptography package supplies P-256 ECDSA device and fog signatures; matplotlib draws the benchmark figures. The root registry uses local JSON files.

Phase 1 Device registration and batch formation

The `protected_connection.py` run connects simulated devices to the fog over TCP on 127.0.0.1 with TLS. The client validates a certificate for localhost from a temporary authority provisioned for the demo. Inside that encrypted connection, the fog sends a random challenge and the device returns HMAC-SHA-256 using its provisioned PSK. A wrong PSK was denied before the device joined the batch.

After initial authentication, each device supplies a DID and DER-encoded P-256 public key. The fog issues another fresh challenge. The device signs it with its private key, and the fog verifies the signature using the submitted public key. The fog then calculates the device leaf and queues it without rebuilding the whole tree.

```
Device leaf = SHA256(UTF8(DID) || DER(public key))
Parent hash = SHA256(0x01 || left child || right child)
```

One root for the completed batch

- At the end of the window, sort all leaf hashes in a fixed order.
- Build the Merkle tree; duplicate the final node at an odd level.
- Store one root under the new epoch in `runtime/roots.json`.
- Save each device's sibling path and epoch in `runtime/proofs.json`.

A verifier recomputes the leaf from the received DID and public key, walks the left/right sibling path, and compares the reconstructed value to the trusted root for that epoch. Old roots remain recorded as historical evidence. They do not, by themselves, mean the device is still allowed access.

Observed run

```
Wrong PSK over TLS: DENY (PSK proof failed)
Initial authentication: PASS; Proof of possession: PASS
TLS TLSv1.3: verified fog certificate for localhost
Finalized epoch: 5 active devices, one trusted root
```

The root registry is a trusted local JSON file, as permitted for the single-zone assignment. This implementation does not claim a blockchain consensus mechanism or multi-fog coordination.

Phase 2 Temporary token access

The fog issues a signed token only after PSK authentication and ECC proof of ownership. Its claims contain a random token ID, zone, DID, public key, fog-side role, and expiry time. The example token lasts 30 seconds. A device waiting in an open batch can write its sensor readings, but a motor controller cannot STOP the production line with provisional access.

For every temporary request, the fog issues a fresh nonce. The device signs the token ID, identity, exact resource and operation, and nonce. The fog checks its own token signature, the expiry, token and device revocation, unused nonce, the device signature, and the provisional policy. A copied token is useless to a different key holder.

Check	Example result
Valid sensor request	WRITE temperature_readings: ALLOW
Copied token	Wrong device private key: DENY
Expired or revoked token	Current status check: DENY
Temporary motor request	STOP production_line: DENY

Phase 3 Permanent proof and authorization

After finalization, a temporary token is rejected. The device submits its DID, public key, epoch, and inclusion proof. The fog verifies the Merkle root, checks its trusted record and revocation list, issues a fresh nonce, verifies the device's signature, and finally applies its resource rule. In this implementation, the live signed request performs nonce and signature checks before the access policy decision.

Fog role	Resource and action	Decision
Temperature sensor	WRITE temperature_readings	ALLOW
Temperature sensor	STOP production_line	DENY
Motor controller	STOP production_line	ALLOW

Sensor temporary WRITE over TLS: ALLOW
 Temporary token after finalization: DENY
 Motor STOP with proof and signature over TLS: ALLOW
 Sensor STOP denied by policy over TLS: DENY

Revocation and security testing

The fog keeps revoked device IDs and token IDs separately. It rereads current device status before issuing a nonce and before allowing a signed request. A request signed before revocation was denied after the fog revoked the device. The next active epoch excluded that device. Its earlier inclusion proof still verified against the earlier root, which illustrates why proof validity and current permission are separate checks.

Old historical Merkle proof: PASS
 Already signed request after revocation: DENY (device revoked)
 New nonce after fog restart: DENY (device revoked)
 Revoked sensor in next epoch: EXCLUDED

Attack demonstrations

Attempt	Expected and observed result	Detection point
Replay an exact signed request	DENY, nonce already used	Fog request validator
Change DID or Merkle leaf	DENY, root reconstruction fails	Fog proof verifier
Copy a valid token to another key	DENY, device signature fails	Fog key ownership check
Change signed token role	DENY, fog token signature fails	Fog token validator
Use expired or revoked token	DENY, current status fails	Fog token and status checks

Decision boundary

An inclusion proof says a DID and key belonged to a historical batch. It does not prove current possession of the private key, current token validity, or permission for a requested operation. This is why the system checks nonce, ECC signature, revocation and policy after verifying the proof. Client-supplied device_id and role fields cannot elevate a request; the fog uses its own record.

Performance evaluation

The performance.py benchmark ran with Python 3.12.14, cryptography 46.0.0 on Linux x86_64 at five device counts: 5, 10, 25, 50 and 100. It repeated each size five times and reported the mean. Registration timing includes device key creation, PSK check, ECC challenge signing and verification, and batch queueing. Batch processing includes sorting leaves and building one root. Proof generation is measured separately.

For each trial, the benchmark verified every proof 100 times, then calculated mean time per proof and verification throughput. It also timed 30 temporary and 30 permanent signed requests. Console registration logs were suppressed during timing. The raw trial CSV and summary CSV are in results/. The table and charts below use the summary CSV; timing will vary with CPU load.

Devices	Batch total ms	Verify ms per proof	Proofs per second
5	1.627	0.00163	614,380
10	1.661	0.00232	445,156
25	3.964	0.00257	389,424
50	8.207	0.00305	328,384
100	15.644	0.00360	278,039

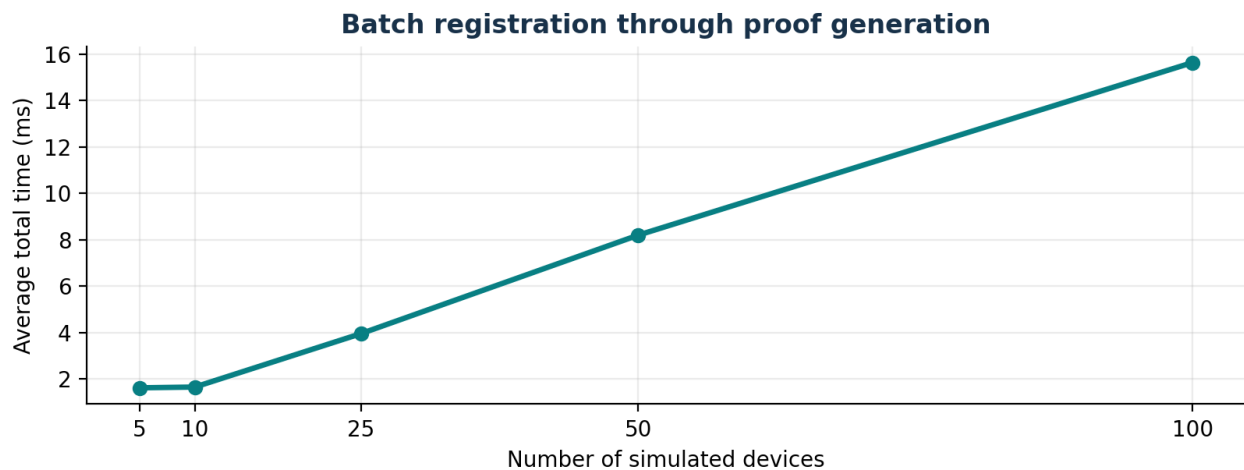


Figure 2 Mean onboarding, one batch build, and proof generation over five runs

The 100-device run took 15.644 ms in total, versus 1.627 ms for five devices. Small changes between nearby device counts are expected when the operating system and background tasks affect short measurements.

Verification latency and throughput

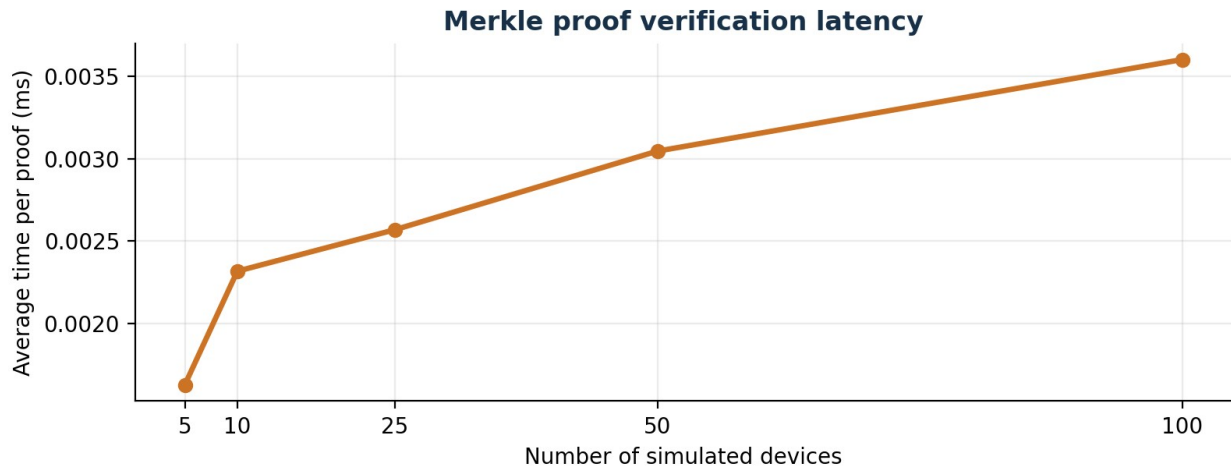


Figure 3 Mean Merkle proof verification time per device

Verification stayed at or below 0.00360 ms per proof in these measurements. A larger tree normally has a longer proof path; small changes between adjacent sizes are also affected by scheduling and timing noise.

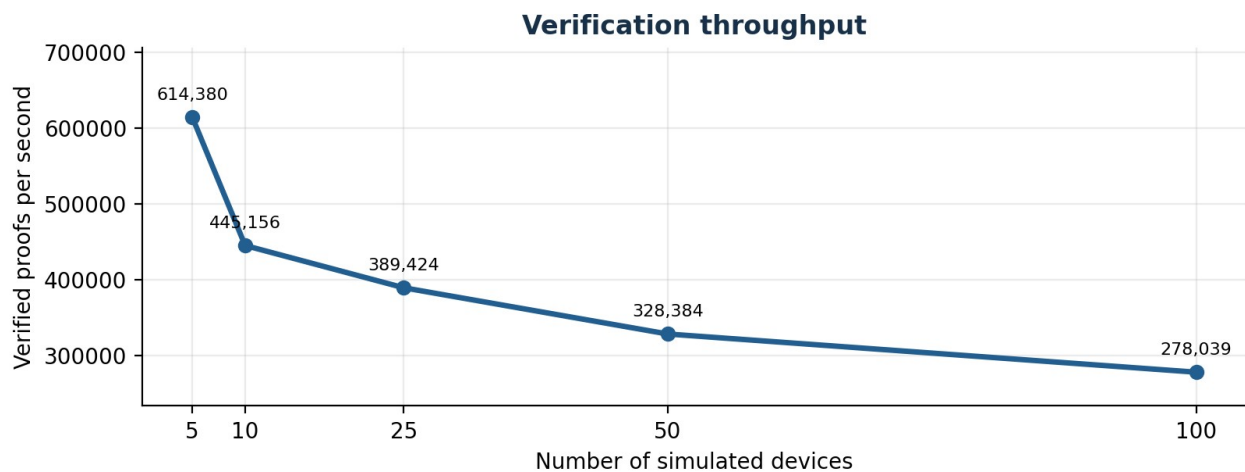


Figure 4 Mean verified proofs per second from the benchmark CSV

Measured throughput was 614,380 proofs per second at five devices and 278,039 at 100 devices. These figures measure the local verifier loop, not end-to-end TLS network throughput. The exact numbers are machine-dependent; the CSV files contain the other measured metrics.

Batch compared with individual registration

The benchmark compares one sorted tree build after the batch closes with a simple approach that rebuilds a sorted tree after every new device arrives. Both use the same device leaves. At N arrivals, the individual approach performs N root calculations; the batch approach performs one. The comparison intentionally omits the extra cost of updating old device proofs and anchoring every intermediate root, so it is conservative.

Measure	Individual arrivals	Batch approach
Root updates for N devices	N	1
Existing proofs	May need new paths as root changes	Stable for the epoch
Tree work	Sort and rebuild repeatedly	Sort and build once
Root registry	Potentially N root versions	One main epoch root

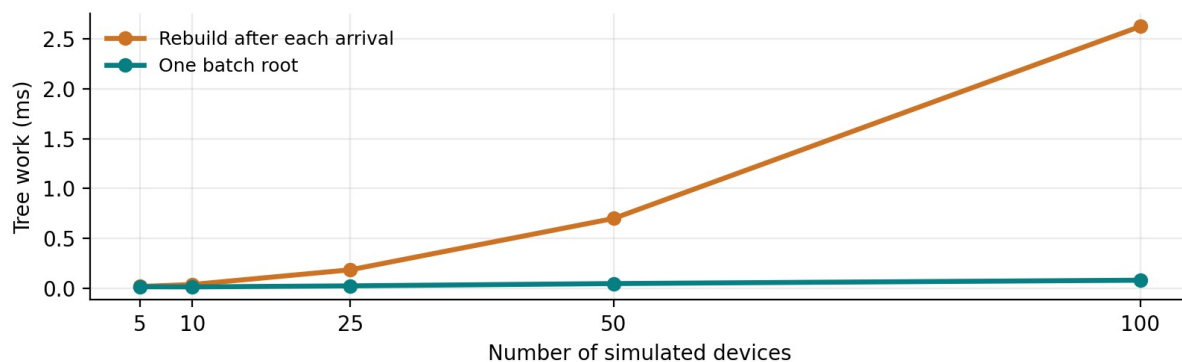


Figure 5 Repeated tree rebuilding versus one batch tree build

Problems and design decisions

- Windows PowerShell and a local Python virtual environment were used; Linux was not required.
- Short proof checks varied between runs; we checked every proof 100 times per trial and averaged five trials to reduce timing noise.
- The protected connection binds only to 127.0.0.1. The demo CA and server key are generated for each run and removed afterward; production deployment would need managed, persistent trust material.
- Device private keys are kept in memory during each run. Historical roots and proofs persist, but an old proof cannot bypass current revocation.
- The project uses a local trusted root registry instead of a blockchain because Assignment 1 permits that choice for one fog and one zone.

Individual accountability

The following test allocation names all three members. File authorship must match the repository's commit history before final submission.

Member	Test explanation responsibility
Muhammad Hassaan Siddiqui	TLS, PSK/ECC onboarding, temporary and permanent allowed requests.
Muhammad Huzaifa	Merkle root and proof, tampered DID, replay and stolen-token checks.
Irbaz	Expiry, device/token revocation, next-epoch exclusion, performance results.

Conclusion

The single-zone simulation completed device onboarding, temporary bridging, batch proof creation, signed permanent access, revocation, and denial of multiple attacks. The performance results show how total registration work grows with the device count while each proof remains quick to check. The scope is one fog and local trust; multi-fog consensus and deployment key management are outside this assignment.